GenPackageDoc

v. 0.43.0

Holger Queckenstedt

17.03.2026

Contents

Chapter 1

Introduction

The Python package **GenPackageDoc** generates the documentation of Python modules. The content of this documentation is taken out of the docstrings of functions, classes and their methods. All docstrings have to be written in reStructuredText (RST) format, that is a certain markdown dialect.

It is possible to extend the documentation by the content of additional files either in reStructuredText format or in LaTeX format.

The documentation is generated in four steps:

1. Files in LaTeX format are taken over immediately.
2. Files in reStructuredText format are converted to LaTeX files.
3. All docstrings of all Python modules in the package are converted to LaTeX files.
4. All LaTeX files together are converted to a single PDF document. This requires a separately installed LaTeX distribution (recommended: TeX Live). A LaTeX distribution is **not** part of **GenPackageDoc** and has to be installed separately!

The sources of **GenPackageDoc** are available in the following GitHub repository:

python-genpackagedoc

The repository **python-genpackagedoc** uses it's own functionality to document itself and the contained Python package **GenPackageDoc**.

Therefore the complete repository can be used as an example about writing a package documentation.

It has to be considered, that the main goal of **GenPackageDoc** is to provide a toolchain to generate documentation out of Python sources that are stored within a repository, and therefore we have dependencies to the structure of the repository. For example: Configuration files with values that are specific for a repository, should not be installed. Such a specific configuration value is e.g. the name of the package or the name of the PDF document.

The impact is: There is a deep relationship between the repository containing the sources to be documented, and the sources and the configuration of **GenPackageDoc** itself. Therefore some manual preparations are necessary to use **GenPackageDoc** also in other repositories.

How to do this is explained in detail in the next chapters.

The outcome of all preparations of **GenPackageDoc** in your own repository is a PDF document like the one you are currently reading.

Chapter 2

Description

2.1 Repository content

What is the content of the repository **python-genpackagedoc**?

- Folder `GenPackageDoc`

  Contains the package code.

  This folder is specific for the package.

- Folder `packagedoc`

  Contains all package documentation related files, e.g. the **GenPackageDoc** configuration, additional input files and the generated documentation itself.

  This folder is specific for the documentation.

- Repository root folder

  - `genpackagedoc.py`
    Python script to start the documentation build
  - `build_backend.py`
    Custom build backend to execute additional installation steps. Currently this is a pattern only reserved for future development.
  - `pyproject.toml`
    Main configuration file for the installation of the component
  - `dump_repository_config.py`
    Little helper to dump the repository configuration to console

2.2 Documentation build process

How do the files and folders listed above, belong together? What is the way, the information flows when the documentation is generated?

- The process starts with the execution of `genpackagedoc.py` within the repository root folder.
- `genpackagedoc.py` creates a repository configuration object

  `config/CRepositoryConfig.py`

- The repository configuration object reads the static repository configuration values out of the TOML file

  `pyproject.toml`

  Exception: The component version is taken from the Python source code

  `GenPackageDoc/version.py`

- The repository configuration object adds dynamic values (like operating system specific settings and paths) to the repository configuration.
- The configuration file `packagedoc_config.json` contains settings like
  - Paths to Python packages to be documented
  - Paths and names of additional RST files
  - Path and name of output folder (LaTeX files and output PDF file)
  - User defined parameter (that can be defined here as global runtime variables and can be used in any RST code)
  - Basic settings related to the output PDF file (like document name, name of author, ...)
  - Path to LaTeX compiler
    (*a LaTeX distribution is not part of* **GenPackageDoc**)

  Be aware of that the within `packagedoc_config.json` specified output folder

  `"OUTPUT" : "./build"`

  will be deleted at the beginning of the documentation build process! Make sure that you do not have any files inside this folder opened when you start the process. In case of the path is relative, the reference is the position of `genpackagedoc.py`. The complete path is created recursively.

  Further details are explained within the json file itself.

- `genpackagedoc.py` also creates an own configuration object

  `GenPackageDoc/CPackageDocConfig.py`

  `CPackageDocConfig.py` takes over all repository configuration values, reads in the static **GenPackageDoc** configuration (`packagedoc_config.json`) and adds dynamically computed values like the full absolute paths belonging to the documentation build process. Also all command line parameters are resolved and checked.

  The reference for all relative paths is the position of `genpackagedoc.py` (that is the repository root folder).

After the execution of `genpackagedoc.py` the resulting PDF document can be found under the specified name within the specified output folder (`"OUTPUT"`). This folder also contains all temporary files generated during the documentation build process.

Because the output folder is a temporary one, the PDF document is copied to the folder containing the package sources and therefore is included in the package installation. This is defined in the **GenPackageDoc** configuration, section `"PDFDEST"`.

Command line

Some configuration parameter predefined within `packagedoc_config.json`, can be overwritten in command line.

`--output`

Path and name of folder containing all output files.

`--pdfdest`

Path and name of folder in which the generated PDF file will be copied to (after this file has been created within the output folder).

Caution: The generated PDF file will per default be copied to the package folder within the repository. This is defined in `packagedoc_config.json`. The version of the PDF file within the package folder will be part of the installation. When you change the PDF destination, then you get this file at another location - but this file will not be part of the installation any more. Installed will be the version, that is still present within the package folder of the repository. Please try to get the bottom of your motivation when you change this setting.

`--configdest`

Path and name of folder in which a dump of the current configuration will be copied to.

The configuration dump is part of the build output (section 'OUTPUT') and available in txt and in json format. It might be useful for further processes to have access to all details regarding the current documentation build.

`--strict`

If `True`, a missing LaTeX compiler aborts the process, otherwise the process continues.

`--simulateonly`

If `True`, the LaTeX compiler is switched off. No new PDF output will be generated. Already existing PDF output will not be updated. This is not handled as error and also not handled as warning. Only the source files will be parsed. This switch is useful to do a pre check for possible syntax issues within the source files without spending time for rendering PDF files.

Example

All listed parameters are optional. **GenPackageDoc** creates the complete output path (`--output`) recursively. Other destination folder (`--pdfdest` and `--configdest`) have to exist already.

2.3 PDF document structure

How is the resulting PDF document structured? What causes an entry within the table of content of the PDF document?

In the following we use terms taken over from the LaTeX world: *chapter*, *section* and *subsection*.

A *chapter* is the top level within the PDF document; a *section* is the level below *chapter*, a *subsection* is the level below *section*.

The following assignments happen during the generation of a PDF document:

- The content of every additionally included separate RST file is a *chapter*.
  - In case of you want to add another chapter to your documentation, you have to include another RST file.
  - The headline of the chapter is the name of the RST file (automatically).
    Therefore the heading within an RST file has to start at section level!
- The content of every included Python module is also a *chapter*.
  - The headline of the chapter is the name of the Python module (automatically).
    This means also that within the PDF document structure every Python module is at the same level as additionally included RST files.
- Within additionally included separate RST files sections and subsections can be defined by the following RST syntax elements for headings:
  - A line underlined with "=" characters is a section
  - A line underlined with "-" characters is a subsection
- Within the docstrings of Python modules the headings are added automatically (for functions, classes and methods)
  - Classes and functions are listed at section level (both classes and functions are assumed to be at the same level).
  - Class methods are listed at subesction level.

  Further nestings of headings are not supported (because we do not want to overload the table of content).

2.4 Examples

2.4.1 Example 1: RST file

The text of this chapter is taken over from an RST file named `Description.rst`.

This RST file contains the following headlines:

```
Repository content
==================

Documentation build process
===========================

PDF document structure
======================

Examples
========

Example 1: RST file
-------------------

Example 2: Python module
------------------------
```

Because `Description.rst` is the second imported RST file, the chapter number is 2. The chapter headline is "Description" (the name of the RST file). The top level headlines *within* the RST file are at *section* level. The fourth section (Examples) contains two subsections.

The outcome is the following part of the table of content:

2.4.2 Example 2: Python module

Part of this documentation is a Python module with name `CDocBuilder.py` (listed in table of content at *chapter* level). This module contains a class with name `CDocBuilder` (listed in table of content at *section* level). The class `CDocBuilder` contains a method with name `Build` (listed in table of content at *subsection* level).

This causes the following entry within the table of contents:

2.5 Interface and module descriptions

How to describe an interface of a function or a method? How to describe a Python module?

To have a unique look and feel of all interface descriptions, the following style is recommended:

Example

```
    """
Description of function or method.

**Arguments:**

* ``input_param_1``

  / *Condition*: required / *Type*: str /

  Description of input_param_1.

* ``input_param_2``

  / *Condition*: optional / *Type*: bool / *Default*: False /

  Description of input_param_2.

**Returns:**

* ``return_param``

  / *Type*: str /

  Description of return_param.
    """
```

Some of the special characters used within the interface description, are part of the RST syntax. They will be explained in one of the next sections.

The docstrings containing the description, have to be placed directly in the next line after the `def` or `class` statement.

It is also possible to place a docstring at the top of a Python module. The exact position doesn't matter - but it has to be the first constant expression within the code. Within the documentation the content of this docstring is placed before the interface description and should contain general information belonging to the entire module.

The usage of such a docstring is an option.

2.6 Runtime variables

What are "runtime variables" and how to use them in RST text?

All configuration parameters of **GenPackageDoc** are taken out of four sources:

1. the static repository configuration
   `pyproject.toml`
2. the dynamic repository configuration
   `config/CRepositoryConfig.py`
3. the static **GenPackageDoc** configuration
   `packagedoc/packagedoc_config.json`
4. the dynamic **GenPackageDoc** configuration
   `GenPackageDoc/CPackageDocConfig.py`

Some of them are runtime variables and can be accessed within RST text (within docstrings of Python modules and also within separate RST files).

This means it is possible to add configuration values automatically to the documentation.

This happens by encapsulating the runtime variable name in triple hashes. This "triple hash" syntax is introduced to make it easier to distinguish between the json syntax (mostly based on curly brackets) and additional syntax elements used within values of json keys.

The name of the repository e.g. can be added to the documentation with the following RST text:

```
The name of the repository is ###REPOSITORYNAME###.
```

This document contains a chapter "Appendix" at the end. This chapter is used to make the repository configuration a part of this documentation and can be used as example.

Additionally to the predefined runtime variables a user can add own ones.
See `"PARAMS"` within `packagedoc_config.json`.

All predefined runtime variables are written in capital letters. To make it easier for a developer to distinguish between predefined and user defined runtime variables, all user defined runtime variables have to be written in small letters completely.

Also the `"DOCUMENT"` keys within `packagedoc_config.json` are runtime variables.

Also within `packagedoc_config.json` the triple hash syntax can be used to access repository configuration values.

With this mechanism it is e.g. possible to give the output PDF document automatically the name of the package:

```
"DOCUMENT" : {
             "OUTPUTFILENAME" : "###PACKAGENAME###.tex",
```

Within parts of the documentation that are written in LaTeX directly, two auto generated LaTeX commands can be used to insert the name of the repository and the name of the package. Both values are taken out of the repository configuration.

1. `\repo` : name of the repository
2. `\pkg` : name of the package

Example:

```
The repository \repo\ contains the package \pkg.
```

Consider the trailing backslash at the end of the command (that together with the following blank indicates a masked blank). This is necessary when you use the command in the middle of a text.

2.7 Syntax aspects

2.7.1 Common rules

Important to know about the syntax of Python and RST is:

- In both Python and RST the indentation of text is part of the syntax!
- The indentation of the triple quotes indicating the beginning and the end of a docstring has to follow the Python syntax rules.
- The indentation of the content of the docstring (= the interface description in RST format) has to follow the RST syntax rules. To avoid a needless indentation of the text within the resulting PDF document and to avoid further unwanted side effects caused by improper indentations, it is strongly required to start at least the first line of a docstring text within the first column! And this first line is the reference for the indentation of further lines of the current docstring. The indentation of these further lines depends on the RST syntax element that is used here.
- In RST also blank lines are part of the syntax!

Why is a proper indentation of the docstrings so much important?

The contents of all doctrings of a Python module will be merged to one single RST document (internally by **GenPackageDoc**). In this single RST document we do not have separated docstring lines any more. We have one text! And we have a relationship between previous lines and following lines in this text. The indentation of these previous and following lines must fit together – accordingly to the RST syntax rules. Otherwise we either get syntax issues during computation or we get text with a layout that does not fit to our expectation.

Please be attentive while typing your documentation in RST format!

2.7.2 Syntax extensions

GenPackageDoc extends the RST syntax by the following topics:

- *newline*

  A newline (line break) is realized by a slash ('`/`') at the end of a line containing any other RST text (this means: the slash must **not** be the only character in line).
  Internally this slash is mapped to the LaTeX command `\newline`.

- *vspace*

  An additional vertical space (size: the height of the '`x`' character - depending on the current type and size of font) is realized by a single slash ('`/`'). This slash must be the only character in line!
  Internally this slash is mapped to the LaTeX command `\vspace{1ex}`.

- *newpage*

  A newpage (page break) is realized by a double slash ('`//`'). These two slashes must be the only characters in line!
  Internally this double slash is mapped to the LaTeX command `\newpage`.

These syntax extensions can currently be used in separate RST files only and are not available within docstrings of Python modules.

2.8 Listings

GenPackageDoc supports code listings and console listings. These listings are available as text block and as inline text also. You can use these listings in RST files and also in LaTeX files immediately. The RST elements for listings are mapped to the corresponding LaTeX commands when **GenPackageDoc** internally converts the RST files in LaTeX files.

Text blocks are realized by markdown directives. Inline text is realized by markdown rules.

Examples:

This is a markdown directive `pythoncode` code example:

```
for index in list:
    print("index")
```

Written in RST format:

```
.. pythoncode::

   for index in list:
       print("index")
```

Written in LaTeX format:

```
\begin{pythoncode}
    for index in list:
        print("index")
\end{pythoncode}
```

This is a markdown directive `robotcode` code example:

```
FOR    ${index}    IN RANGE    0    ${max}
    log    index: ${index}    console=yes
END
```

Written in RST format:

```
.. robotcode::

   FOR    ${index}    IN RANGE    0    ${max}
       log    index: ${index}    console=yes
   END
```

Written in LaTeX format:

```
\begin{robotcode}
   FOR    ${index}    IN RANGE    0    ${max}
       log    index: ${index}    console=yes
   END
\end{robotcode}
```

This is a markdown directive `pythonlog` code example:

```
index: 0
index: 1
index: 2
```

Written in RST format:

```
.. pythonlog::

   index: 0
   index: 1
   index: 2
```

Written in LaTeX format:

```
\begin{pythonlog}
   index: 0
   index: 1
   index: 2
\end{pythonlog}
```

This is a markdown directive `robotlog` code example:

```
index: 0
index: 1
index: 2
```

Written in RST format:

```
.. robotlog::

   index: 0
   index: 1
   index: 2
```

Written in LaTeX format:

```
\begin{robotlog}
   index: 0
   index: 1
   index: 2
\end{robotlog}
```

This is inline Python code: `print("Hello Python")`
Written in RST format:

```
:pcode:`print("Hello Python")`
```

Written in LaTeX format:

```
\pcode{print("Hello Python")}
```

This is inline Robot code: `log  Hello RobotFramework AIO`
Written in RST format:

```
:rcode:`log~~~~Hello RobotFramework AIO`
```

Written in LaTeX format:

```
\rcode{log~~~~Hello RobotFramework AIO}
```

Be aware of the tilde '~'! LaTeX per default reduces multiple blanks to one single blank. **The tilde is required here to keep multiple blanks.**

This is inline Python log: `Hello Python`
Written in RST format:

```
:plog:`Hello Python`
```

Written in LaTeX format:

```
\plog{Hello Python}
```

This is inline Robot log: `Hello RobotFramework AIO`
Written in RST format:

```
:rlog:`Hello RobotFramework AIO`
```

Written in LaTeX format:

```
\rlog{Hello RobotFramework AIO}
```

2.9 Diagrams

A *diagram* in this context is a picture that is rendered out of source code. **GenPackageDoc** supports **PlantUML** that supports a wide range of diagrams.

To use the **PlantUML** functionality with **GenPackageDoc**, some preconditions have to be fulfilled:

1. **PlantUML** is installed (either as stand-alone installation or as VSCodium extension)
2. **GenPackageDoc** is configured (`packagedoc_config.json`): a. In section `"DIAGRAMS"` a path to a diagrams folder is defined (containing the diagrams source code). b. In section `"JAVA"` path and name of the java interpreter is defined (because **PlantUML** is a java application). c. In section `"PLANT_UML"` path and name of the **PlantUML** application is defined.

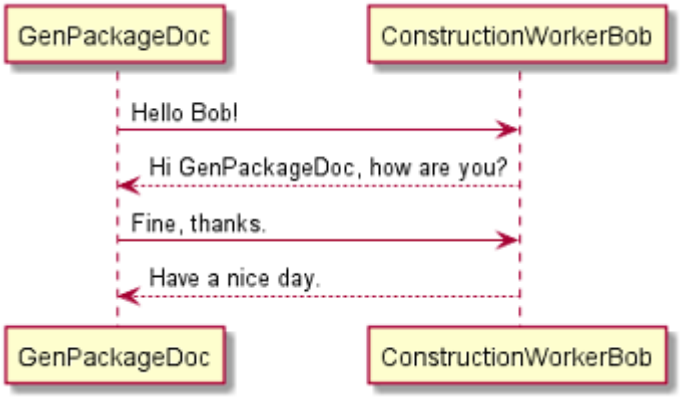
All **PlantUML** source code files within the `"DIAGRAMS"` folder need to have the extension `puml`.

2.9.1 Example 1: Sequence diagram

The following code of a `puml` file produces a sequence diagram:

```
@startuml
GenPackageDoc -> ConstructionWorkerBob: Hello Bob!
ConstructionWorkerBob --> GenPackageDoc: Hi GenPackageDoc, how are you?
GenPackageDoc -> ConstructionWorkerBob: Fine, thanks.
ConstructionWorkerBob --> GenPackageDoc: Have a nice day.
@enduml
```

Result:



2.9.2 Example 2: JSON diagram

The following code of a `puml` file produces a sequence diagram:

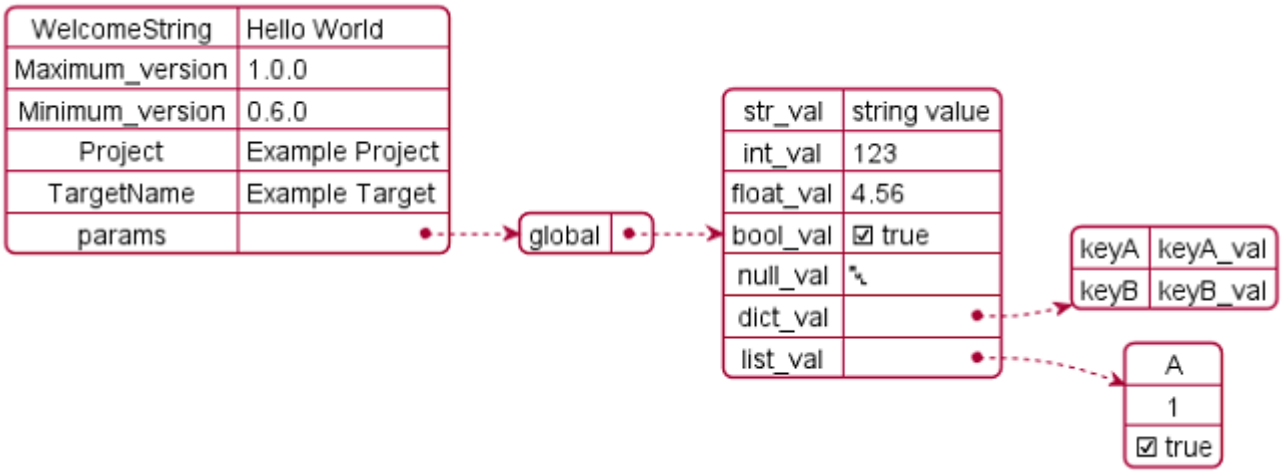
```
@startjson
{
  "WelcomeString"   : "Hello World",

  "Maximum_version" : "1.0.0",
  "Minimum_version" : "0.6.0",

  "Project"         : "Example Project",
  "TargetName"      : "Example Target",

  "params" : {
              "global" : {
                          "str_val"   : "string value",
                          "int_val"   : 123,
                          "float_val" : 4.56,
                          "bool_val"  : true,
                          "null_val"  : null,
                          "dict_val"  : {"keyA" : "keyA_val", "keyB" : "keyB_val"},
                          "list_val"  : ["A", 1, true]}
             }
}
@endjson
```

Result:



2.9.3 Picture import

In case of **PlantUML** is configured in the **GenPackageDoc** configuration and in case of `puml` files are available within the diagrams folder, **GenPackageDoc** automatically calls **PlantUML** to render the diagrams. They can be imported in the following way:

1. Import in RST code:

   ```
   .. image:: ./diagrams/SequenceDiagram.png
   ```

2. Import in LaTeX code:

   ```
   \includegraphics[scale=0.7]{./diagrams/SequenceDiagram.png}
   ```

The user needs to adapt the scaling to make the rendered diagrams fit to a page of a PDF document in best way. But this scaling only works in LaTeX code, but not in RST code.

Chapter 3

CDocBuilder.py

Python module containing all methods to generate tex sources.

3.1 Class: CDocBuilder

Main class to build tex sources out of docstrings of Python modules and separate text files in rst format.

Depends on a json configuration file, provided by a `oPackageDocConfig` object (this includes the Repository configuration).

Method to execute: `Build()`

3.1.1 Method: Build

Arguments:

(*no arguments*)

Returns:

- `bSuccess`
  / *Type*: bool /
  Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
  / *Type*: str /
  The result of the computation of the method `sMethod`.

Chapter 4

CInterface.py

Python module containing an interface for **GenPackageDoc**. This interface can be used to get access to the LaTeX stylesheets that are part of the **GenPackageDoc** installation.

4.1 Class: CInterface

4.1.1 Method: GetLaTeXStyles

The LaTeX stylesheets are part of the installation of **GenPackageDoc**. In case of anyone else than **GenPackageDoc** needs these stylesheets, this method can be used to copy them to any other folder.

Arguments:

- `sDestination`
  / *Condition*: required / *Type*: str /
  Path and name of a folder in which the styles folder from **GenPackageDoc** will be copied.

Returns:

- `bSuccess`
  / *Type*: bool /
  Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
  / *Type*: str /
  The result of the computation of the method `sMethod`.

Chapter 5

CPackageDocConfig.py

Python module containing the configuration for **GenPackageDoc**. This includes the repository configurantion and command line values.

5.1 Function: printerror

5.2 Class: CPackageDocConfig

5.2.1 Method: PrintConfig

Prints all cofiguration values to console.

5.2.2 Method: PrintConfigKeys

Prints all cofiguration key names to console.

5.2.3 Method: Get

Returns the configuration value belonging to a key name.

5.2.4 Method: GetConfig

Returns the complete configuration dictionary.

Chapter 6

CPatterns.py

Python module containing source patterns used to generate the tex file output.

6.1 Class: CPatterns

The `CPatterns` class provides a set of LaTeX source patterns used to generate the tex file output.

All source patterns are accessible by corresponding `Get` methods. Some source patterns contain placeholder that will be replaced by input parameter of the `Get` method.

6.1.1 Method: GetHeader

Defines the header of the main tex file.

Arguments:

- `sTitle`
  / *Condition*: required / *Type*: str /
  The title of the output document (name of the described package)
- `sVersion`
  / *Condition*: required / *Type*: str /
  The version of the output document (version of the described package)
- `sAuthor`
  / *Condition*: required / *Type*: str /
  The author of the output document (author of the described package)
- `sDate`
  / *Condition*: required / *Type*: str /
  The date of the output document (date of the described package)

Returns:

- `sHeader`
  / *Type*: str /
  LaTeX code containing the header of main tex file.

6.1.2 Method: GetChapter

Defines single chapter of the main tex file.

A single chapter is equivalent to an additionally imported text file in rst format or equivalent to a single Python module within a Python package.

Arguments:

- `sHeadline`

  / *Condition*: required / *Type*: str /

  The chapter headline (that is either the name of an additional rst file or the name of a Python module).

- `sLabel`

  / *Condition*: required / *Type*: str /

  The chapter label (to enable linking to this chapter)

- `sDocumentName`

  / *Condition*: required / *Type*: str /

  The name of a single tex file containing the chapter content. This file is imported in the main text file after the chapter headline that is set by `sHeadline`.

Returns:

- `sHeader`

  / *Type*: str /

  LaTeX code containing the headline and the input of a single tex file.

6.1.3 Method: GetFooter

Defines the footer of the main tex file.

Arguments:

(*no arguments*)

Returns:

- `sFooter`

  / *Type*: str /

  LaTeX code containing the footer of the main tex file.

6.1.4 Method: GetAutodefinedHeader

Defines the header of the autodefined LaTeX sty file.

Arguments:

(*no arguments*)

Returns:

- `sAutodefinedHeader`

  / *Type*: str /

  LaTeX code containing the header of the autodefined LaTeX sty file.

Chapter 7

CSourceParser.py

Python module containing all methods to parse the documentation content of Python source files.

7.1 Class: CSourceParser

The `CSourceParser` class provides a method to parse the functions, classes and their methods together with the corresponding docstrings out of Python modules. The docstrings have to be written in rst syntax.

7.1.1 Method: ParseSourceFile

The method `ParseSourceFile` parses the content of a Python module.

Arguments:

- `sFile`
  / *Condition*: required / *Type*: str /
  Path and name of a single Python module.
- `bIncludePrivate` (currently not active, is `False`)
  / *Condition*: optional / *Type*: bool / *Default*: False /
  If `False`: private methods are skipped, otherwise they are included in documentation.
- `bIncludeUndocumented`
  / *Condition*: optional / *Type*: bool / *Default*: True /
  If `True`: also classes and methods without docstring are listed in the documentation (together with a hint that information is not available), otherwise they are skipped.

Returns:

- `dictContent`
  / *Type*: dict /
  A dictionary containing all the information parsed out of `sFile`.
- `bSuccess`
  / *Type*: bool /
  Indicates if the computation of the method `sMethod` was successful or not.
- `sResult`
  / *Type*: str /
  The result of the computation of the method `sMethod`.

Chapter 8

markdown_extensions.py

Python module containing markdown extensions: Markdown directives and markdown roles mapped to LaTeX code listings and console listings.

8.1 Function: pcode_role

8.2 Function: rcode_role

8.3 Function: plog_role

8.4 Function: rlog_role

8.5 Class: CustomLaTeXTranslator

Mapping between markdown and LaTeX w.r.t.:

- Python code listings
- Robot code listings
- Python log listings
- Robot log listings

8.5.1 Method: visit_literal_block

8.5.2 Method: visit_literal

8.6 Class: CustomLaTeXWriter

8.7 Class: PythonCodeDirective

8.7.1 Method: run

8.8 Class: RobotCodeDirective

8.8.1 Method: run

8.9 Class: PythonLogDirective

8.9.1 Method: run

8.10 Class: RobotLogDirective

8.10.1 Method: run

Chapter 9

Appendix

About this package:

Table 9.1: Package setup

Setup parameter	**Value**
Name	GenPackageDoc
Version	0.43.0
Date	17.03.2026
Description	Documentation builder for Python packages
Package URL	python-genpackagedoc
Author	Holger Queckenstedt
Email	Holger.Queckenstedt@de.bosch.com
License	Apache-2.0
Python required	>=3.11

Chapter 10

History

Version 0.43.0 (17.03.2026)

Maintenance:

- Usage of **Setuptools** replaced by usage of **PIP/TOML/Setuptools**
- History reworked (youngest verion on top now) and migrated from LaTeX format to RST format

Added:

- Possibility to exclude interface files from computation
- Markdown directives to support Python and **Robot Framework** code listings and log listings

Version 0.42.0 (12.01.2026)

Maintenance:

- Usage of **pandoc/pypandoc** replaced by usage of **docutils**

Version 0.41.0 (11.07.2023)

Added:

- Text macro `\q{}` to set double quotes

Version 0.40.0 (09.05.2023)

Added:

- **PlantUML** support
- Possibility to render diagrams directly from text source code

Version 0.39.0 (06.01.2023)

Added:

- Masking of underlines in case of the content of `\repo` or `\pkg` contains underlines (masking required by LaTeX)

Version 0.38.0 (30.11.2022)

Removed:

- Harming ligatures that were added by **Pandoc** automatically to LaTeX code in case of multiple minus characters in names

Version 0.37.0 (21.11.2022)

Added:

- LaTeX commands `pythonlog` and `plog`
- `keyword` decorator detection

Maintenance:

- LaTeX style adaptions
- `INCLUDEPRIVATE` temporarily switched off (requires bugfixes)

Version 0.36.0 (17.11.2022)

Maintenance:

- Brightness of all colors of listings reduced to 45% (both text boxes and inline)

Version 0.35.0 (16.11.2022)

Maintenance:

- Layout settings of some LaTeX commands adapted
- Repository name and package name in bold
- Inline code and inline listings in clearer colors

Version 0.34.0 (07.11.2022)

Added:

- Auto defined LaTeX style file containing mnemotechnical commands to type the repository name and the package name

Version 0.33.0 (19.09.2022)

Maintenance:

- Rework of label mechanism (to enable unique links to functions, classes and methods with names that are not unique over all Python modules within a package)

Version 0.32.0 (16.09.2022)

Added:

- Labels at `chapter` level

Maintenance:

- Partial rework of label mechanisms

Version 0.31.0 (12.09.2022)

Fixes:

- Import path of a module in PDF file

Version 0.30.0 (31.08.2022)

Added:

- `simulateonly` mode (command line switch to skip the PDF generation)

Version 0.29.0 (24.08.2022)

Changes:

- Layout of import path of a module (in PDF file)

Version 0.28.0 (23.08.2022)

Added:

- LaTeX environment variable `GENDOC_LATEXPATH`

Maintenance:

- File **robotframeworkaio.sty** aligned to version of this file in **GenMainDoc**

Version 0.27.0 (17.08.2022)

Added:

- LaTeX style definition for Python syntax highlighting

Version 0.26.0 (27.07.2022)

Maintenance:

- History reworked

Added:

- File **common.sty** (common LaTeX commands e.g. to support the history)

Version 0.25.0 (27.07.2022)

Maintenance:

- Layout of **RobotFramework AIO** syntax highlighting (in **.sty** files)

Version 0.24.0 (25.07.2022)

Maintenance:

- Line breaks in listings (in **robotframeworkaio.sty**)

Version 0.23.0 (13.07.2022)

Maintenance:

- File **preamble.tex**

Version 0.22.0 (13.07.2022)

Maintenance:

- File **preamble.tex**
- Folder **styles**

Fix:

- **setup.py** installs tex files also

Version 0.21.0 (12.07.2022)

Maintenance:

- Separated file **preamble.tex**

Version 0.20.0 (29.06.2022)

Fix:

- Added missing masking of underlines in PDF document title (required for LaTeX)

Version 0.19.0 (28.06.2022)

Added:

- Method `GetLaTeXStyles`

Maintenance:

- **PythonExtensionsCollection** updated to version 0.8.0

Version 0.18.0 (20.06.2022)

Added:

- Parameter to define an output folder for a dump of final configuration

Version 0.17.0 (17.06.2022)

Added:

- Possibility to dump the configuration

Maintenance:

- Code and error handling

Version 0.16.0 (01.06.2022)

Maintenance:

- Path computation reworked

Version 0.15.0 (31.05.2022)

Added:

- **GenPackageDoc** command line
- Separate **GenPackageDoc** configuration class

Version 0.14.0 (27.05.2022)

Added:

- LaTeX compiler check
- Control parameter `STRICT` added to **GenPackageDoc** configuration

Version 0.13.0 (24.05.2022)

Maintenance:

- LaTeX style definitions moved to a separate folder

Version 0.12.0 (19.05.2022)

Added:

- Admonitions based on LaTeX environment `tcolorbox`

Maintenance:

- Layout adaptions in titlepage

Fix:

- Page numbering in TOC

Version 0.11.0 (18.05.2022)

Added:

- Possibility to import **tex** files

Version 0.10.0 (17.05.2022)

Added:

- Postprocessing for **rst** and **tex** source files added

Fix:

- `multiply-defined labels`

Version 0.9.2 (16.05.2022)

Fix:

- Automated line breaks within code blocks

Version 0.9.1 (11.05.2022)

Maintenance:

- Documentation

Version 0.9.0 (10.05.2022)

Maintenance:

- Layout maintenance and syntax extensions for `newline`, `newpage` and `vspace`

Version 0.8.0 (10.05.2022)

Changes:

- Bugfixes and code maintenance

Added:

- Version history

Version 0.7.0 (10.05.2022)

Added:

- Setup process and file **README.rst**

Maintenance:

- Code

Version 0.6.0 (09.05.2022)

Added:

- Parameter `INCLUDEUNDOCUMENTED`

Version 0.5.0 (09.05.2022)

Added:

- Parameter `INCLUDEPRIVATE`

Version 0.4.0 (06.05.2022)

Added:

- Possibility to describe complete Python modules

Version 0.3.0 (06.05.2022)

Added:

- Automated headings for functions, classes and methods

Version 0.2.0 (05.05.2022)

Added:

- Python syntax highlighting within code blocks

Version 0.1.0 (04/2022)

Initial version

GenPackageDoc in GitHub
GenPackageDoc in PyPi

GenPackageDoc.pdf

Created at 17.03.2026 - 13:49:03

by GenPackageDoc v. 0.43.0 / 17.03.2026